Metadata of the chapter that will be visualized in SpringerLink

Book Title	Graph Transformation	
Series Title		
Chapter Title	User-Defined Types and Operations in **GROOVE**	
Copyright Year	2026	
Copyright HolderName	The Author(s), under exclusive license to Springer Nature Switzerland AG	

Corresponding Author	Family Name	**Rensink**
	Particle	
	Given Name	**Arend**
	Prefix	
	Suffix	
	Role	
	Division	
	Organization	University of Twente
	Address	Enschede, Netherlands
	Email	arend.rensink@utwente.nl
	ORCID	http://orcid.org/0000-0002-1714-6319

Abstract

The graph transformation tool GROOVE supports data manipulation through a signature with fixed sorts `int`, `real`, `bool` and `string` and a collection of pre-defined operators. Graphs can contain algebra values as special nodes; attributes correspond to edges to such value nodes.

This setup has limitations. For instance, it is awkward to model more advanced data calculations—for instance, advanced string or statistical operations. Moreover, the algebraic underpinning means that non-determinism (for instance, randomness) cannot be modelled. Finally, there is no support for structured data types, such as records.

In this paper, we describe an extension of GROOVE that allows *user-defined types and operations*, based on programmable Java classes and methods. Essentially, users can annotate their own classes and methods and make these available to GROOVE at runtime, after which they can be employed in rule systems. Moreover, methods can be declared as *indeterminate*, meaning that their outcome is not completely fixed by their arguments; this allows randomness and other forms of non-determinism.

User-Defined Types and Operations in GROOVE

Arend Rensink(✉)

University of Twente, Enschede, Netherlands
arend.rensink@utwente.nl

Abstract. The graph transformation tool GROOVE supports data manipulation through a signature with fixed sorts `int`, `real`, `bool` and `string` and a collection of pre-defined operators. Graphs can contain algebra values as special nodes; attributes correspond to edges to such value nodes.

This setup has limitations. For instance, it is awkward to model more advanced data calculations—for instance, advanced string or statistical operations. Moreover, the algebraic underpinning means that non-determinism (for instance, randomness) cannot be modelled. Finally, there is no support for structured data types, such as records.

In this paper, we describe an extension of GROOVE that allows *user-defined types and operations*, based on programmable Java classes and methods. Essentially, users can annotate their own classes and methods and make these available to GROOVE at runtime, after which they can be employed in rule systems. Moreover, methods can be declared as *indeterminate*, meaning that their outcome is not completely fixed by their arguments; this allows randomness and other forms of non-determinism.

AQ1

1 Introduction

For graph transformation tools to be able to model practically relevant domains, they have to support a notion of data attributes. In the case of GROOVE, this support is rooted in algebra: there is a pre-defined signature with four primitive sorts (`int`, `real`, `bool` and `string`), with for each sort a set of constants and a collection of pre-defined operators. GROOVE offers a choice of four different algebras at various levels of abstraction (the `term` algebra, in which no computation takes place; the `big` algebra for unbounded-precision arithmetic; the `java` algebra, directly based on Java types; and the `point` algebra, where each carrier sort contains precisely one value). Data values are incorporated in graphs as special *value nodes*, typed by their sorts; attributes are edges from ordinary (non-value) nodes to value nodes. Rules can contain sort-typed variable nodes and expressions built from operators, constants and variable nodes.

This setup has clear limitations. For instance, it is awkward to model advanced data manipulation beyond the pre-defined types and operations; e.g., trigonometrical or statistical calculations. Moreover, the algebraic underpinning

B. Archibald and O. Semeráth (Eds.): ICGT 2026, LNCS 16624, pp. 1–12, 2026.
https://doi.org/10.1007/978-3-032-29730-3_12

means that operations are *determinate*: their outcome is always completely determined by their arguments. This means that, for instance, randomness cannot be modelled. Finally, there is no support for structured data types, such as records; while this does not affect the modelling power *per se*, it does hamper the usability for practical purposes.

In this paper, we describe an extension of GROOVE to *user-defined types and operations*, based on programmable Java classes and methods, that eliminates these limitations. Essentially, rules can invoke methods of appropriately annotated Java classes, made available to GROOVE at runtime. User-defined types are limited to records with primitively typed fields; user-defined operations can have any signature based on the original primitive types plus user types. Moreover, methods can be annotated as *indeterminate*, meaning that their outcome is not completely determined by their arguments. This allows the inclusion of randomness and other forms of non-determinism.

Below, we discuss the choices we made and their rationale (Sect. 2) and we briefly recapitulate the theoretical underpinning of GROOVE attributes (Sect. 3) to show how user-defined types and operations fit into that. After that, we discuss the actual implementation and illustrate it on the basis of a toy example (Sect. 4). Conclusions and related work are presented in Sect. 5.

The functionality described in this paper has been incorporated in GROOVE release 7.5.2, and the Java code and GROOVE rule system of the example are available as part of this paper's repository.

2 Design Considerations and Decisions

The extension reported in this paper was motivated by the lack of convenience, reported by users, in modelling structured data using just graph nodes and primitively typed attributes. Some particular concerns are:

Representation. Using nodes and primitive attributes causes all data values to become part of the graph structure, making them less easy to recognise and represent textually. Essentially, a term such as `Date(6,6,1964)` is more immediately comprehensible than a **`Date`**-typed node with three outgoing **`int`**-typed edges.

Immutability. Manipulating structured data should not modify the existing values but create new ones. For instance, if one adds a year to a given `Date`, this results in a new `Date` and not in the replacement of the `year`-edge. To mimic that behaviour using standard graph transformation, one has to be careful in explicitly creating a new node whenever this occurs.

Equality. For data types, it is unintuitive to have multiple copies of the same value: if data values correspond to graph nodes, equality should correspond to node identity. This is hard to maintain when relying on standard graph nodes, in particular in light of the previous point: rather than blithely creating a new node whenever a new (structured) data value is required, one has to reuse the node with that new value if it exists, and create one otherwise.

Operations. A particular structured data type may come with a set of corresponding operations (in object-oriented programming typically provided as methods) for instance, in the case of a `Date`, one might want to obtain the next date value or to reason about the number of days since the beginning of the year. If the data type is modelled as part of a graph, there is no obvious way to encode and invoke such operations. Note that, since operations offer the possibility to "wrap" any complex calculation, they are convenient for primitive as well as structured data.

A final point at which the attribute support of GROOVE was found to be lacking is unrelated to structured data.

Randomness. All primitive GROOVE operations are deterministic. This makes it very awkward to model systems that may involve randomness or any other kind of indeterminacy in their behaviour. For instance, the only way to model the roll of a die is in modelling all possible outcomes explicitly, meaning that all resulting behaviours are part of the resulting state space.

In addressing these issues, the following questions had to be answered:

- *What kind of data types* should be offered, in addition to the currently available primitive types?
- *How should used-defined data types be integrated* into the existing attribute support architecture?
- *In what language* should users program their data types and operations?

Though these questions are not completely independent, we address them one by one.

Kinds of Data Types. An obvious source of inspiration are the data type structures developed for programming languages; see, e.g., [6]. Two kinds of data structure stand out as being potentially useful: *records* and *lists*. Other kinds of structured data types, such as *sum types* or *object types*, are reasonably compatible with node types and do not urgently require a different, data-specific solution, whereas the use of more powerful notions such as *function types* is less obvious in the context of graph-based modelling.

The current work focusses on record types; moreover, we have chosen to restrict fields to primitive data types. Both support for arrays and for nested records are left as future work.

Integration. The strength of GROOVE lies in the formal foundation of its models, based on typed attributed graphs, and its capability for state space generation, including automatic isomorphism checking. The extension to user-defined structured data means that the typing of attributes has had to be reconsidered. Ideally, the type system should be able to distinguish between different user types and so catch errors in the invocation of user-defined operators on the wrong user-defined type. However, the solution chosen for now is to group all user-defined types under a single algebraic sort `user`: erroneous invocations are caught at runtime by generating a special "user error value".

Language. To allow users full flexibility in their implementations, a natural choice has been to offer Java as source language, given that *(i)* it is also the language of GROOVE itself,*(ii)* the native `record` structure provides a lot of scaffolding,*(iii)* information about the types and operations can be imported through annotations, and *(iv)* invocation can be programmed through reflection.

Moreover, using Java this also enables the reuse of pre-existing code. On the other hand, from a formal reasoning point of view, a disadvantage is that it is impossible to restrict what may happen within user-defined operations: they may easily have side-effects that can lead to unpredictable results, also because there are no guarantees about the precise moment at which GROOVE invokes them. In other words, the correctness of the state space exploration partially depends on user discipline.

3 Theoretical Underpinning

Since this paper is especially meant to introduce a tool extension, we limit the theoretical exposition to the necessary minimum.

3.1 System Types and Operations

The attribute support in GROOVE is based on multi-sorted algebra. This starts with the concept of a *signature*, being a tuple $\langle S, C, O\rangle$ of disjoint sets S (*sorts*), C (*constants*) and O (*operators*); C and S are partitioned into $\{C^s\}_{s\in S}$ and $\{O^s\}_{s\in S}$, respectively. GROOVE offers only primitive sorts, in the form of $S = \{\texttt{bool}, \texttt{int}, \texttt{real}, \texttt{string}\}$; C consists of literals in Java notation; and O is a set of elementary arithmetic and string manipulation operators.

The *sort of* a constant or operator $p \in C \cup O$ is that $s_p \in S$ for which $p \in C^{s_p} \cup O^{s_p}$. Moreover, each $o \in O$ has an *input signature* $\sigma_o \in S^*$. In addition, there is a set of *variables* X, likewise partitioned into $\{X^s\}_{s\in S}$.

Given a signature $\langle S, C, O\rangle$ and a set of variables X, the set of *terms* T, once more partitioned into $\{T^s\}_{s\in S}$, is defined as the smallest set such that *(i)* $c \in T^{s_c}$ for all $c \in C$,*(ii)* $x \in T^{s_x}$ for all $x \in X$, and *(iii)* $o(t_1, \cdots t_{|\sigma_o|}) \in T^{s_o}$ for all $o \in O$, given $t_i \in T^{\sigma_o|_i}$ for all $1 \le i \le |\sigma_o|$.

An *algebra* based on $\langle S, C, O\rangle$ consists of *carrier sets* D^s for all $s \in S$, *values* $v_c \in D^{s_c}$ for all $c \in C$, and *functions* $f_o \colon D_1 \times \cdots \times D_{|\sigma_o|} \to D^{s_o}$ for all $o \in O$, where $D_i = D^{\sigma_o|_i}$ for all $1 \le i \le |\sigma|$. Given a *valuation* $\nu \colon X \to D$ (with $\nu(x) \in D^{s_x}$ for all $x \in X$), every term $t \in T^s$ gives rise to a value in D^s.

Out of the box, GROOVE offers four algebras: `term`, where $D^s = T^s$ for all $s \in S$; `big`, where the carrier sets are essentially the mathematical sets of booleans $\mathbb{B}$, integers $\mathbb{I}$, rationals $\mathbb{Q}$ and strings $\mathbb{S} = \mathbb{C}^*$ (for the set $\mathbb{C}$ of Unicode characters); `java`, where the carrier sets are the values of the Java types **`boolean`**, **`int`**, **`double`** and `String`; and finally `point`, for which all carrier sets are singletons. In addition, `big` and `java` contain special error values $\bot_s \in D^s$ for each sort $s \in S$ to ensure that all f_o are fully defined; for instance, a term t_1/t_2, denoting the division of t_1 by t_2, yields $\bot_{\texttt{int}}$ respectively $\bot_{\texttt{real}}$ if t_2 evaluates to zero.

It may be noted that the above formalisation does not include equations, and thus omits an ingredient that is essential in algebraic reasoning. Indeed, GROOVE does not offer equation-based analysis capabilities. If operators would have equational specifications, then `term` would not be a valid algebra.

We will not go into the formalisation of GROOVE graphs here, except to recall that nodes are taken from a universe that includes the combined carrier sets D (of the selected algebra) as so-called *value nodes*, and edges can have D-nodes as their target (but not their source). Rules can contain terms encoded as AST-like graph structures (see [4] for the encoding) that include variables as nodes. A *match* of a rule r into a host graph establishes a valuation ν_r for r's variable nodes, which is used to evaluate all terms contained in r. Part of the outcome of that evaluation serves as an application condition for r, another part determines the values in the target graph. Any combination of host graph G and match m of r into G that (when evaluated) satisfies r's application conditions gives rise, through the usual algebraic pushout construction (see [2,3]), to a target graph H, which is therefore uniquely determined up to isomorphism. The combined construction is denoted $G \overset{r,m}{\Longrightarrow} H$.

3.2 User Types and Operations

We introduce a single new sort `user`. User-defined operators come in three kinds:

- *Constructors* u, with $\sigma_u \in S^*$ and $s_u = \mathsf{user}$. As the name implies, from a sequence of primitive-sorted input values a constructor returns a unique `user`-sorted value, being essentially the *tuple* of input values—in other words, formally speaking, u is a product operator. Uniqueness here means that the corresponding function f_u is injective and the image sets $\operatorname{img} f_u$ are guaranteed to be disjoint for different constructors u.
- *Accessors* $a_{u,1}, \ldots a_{u,n}$ for every constructor u and $1 \leq i \leq n = |\sigma_u|$, with $\sigma_{a_i} = \mathsf{user}$ and $s_{a_i} = \sigma_u|_i$. The accessor $a_{u,i}$ retrieves the i'th element of the tuple constructed by u; equationally this can be expressed as $a_i(u(x_1, \ldots, x_n)) = x_i$. Outside $\operatorname{img} f_u$, the accessors $a_{u,i}$ are undefined—or rather, they yield $\perp_{\mathsf{user}}$.
- *Other operators*, of any signature and sort.

Let U be the collection of constructors; then the `user`-carrier set is defined by

$$D^{\mathsf{user}} = \bigcup_{u \in U} \operatorname{img} f_u \ .$$

This means that all user-defined values can be obtained through construction. In particular, though there may be "other operators" in $O^{\mathsf{user}} \setminus U$ that also return `user`-sorted values, those can always (alternatively) be produced by some $u \in U$, hence can be understood as tuples of which the elements can be retrieved using accessors.

An important additional feature is that the "other operators" may be designated as *indeterminate*, meaning that their result is actually not fixed by their inputs.

The prototypical example is a random generator, which has an empty input signature (it takes no arguments) but by its very nature produces an unpredictable result. Algebraically, an indeterminate operator $o \in O_\iota$ can be captured by a powerset function $F_o \colon D_1 \times \cdots \times D_{|s|} \to (\mathcal{P}(D^{s_o}) \setminus \{\emptyset\})$, rather than a simple function f_o as above; each time F_o is *invoked* on a set of values $v_1, \ldots, v_{|s|}$, it returns a *single* value from $F_o(v_1, \ldots, v_{|s|})$.

As a consequence, if a rule r contains an indeterminate operator, its actual application is no longer deterministic. Given a host graph G and a match m, the indeterminate operators of r are invoked based on the valuation ν_r, as part of the evaluation of the terms in r. The combined outcome may mean that r's application condition is not satisfied; and if it is satisfied, the graph H produced by the transformation $G \overset{r,m}{\Longrightarrow} H$ is likely to be partially determined by that indeterminate outcome. The overall effect is that explorations of the same rule system at different moments in time can have different results—which is precisely what one would expect from a system that includes a form of randomness.

4 Implementation

User types and operations are defined through annotated Java classes, made available to the JVM upon invoking GROOVE. Annotations come in two flavours:

`@UserType`. This is a class-level annotation, to be used on public **`record`** types with all field types taken from the `java` algebra (**`boolean`**, **`int`**, **`double`** or `String`).

```
@UserType
public record Nm(ty₁ fd₁, ..., tyₙ fdₙ)
```

defines an operator $\mathtt{Nm} \in O^{\mathsf{user}}$ with $\sigma_{\mathtt{Nm}} = s_1 \cdots s_n$ (where the s_i are the sorts corresponding to the Java types $\mathtt{ty}_i$) as well as operators $\mathtt{fd}_i \in O^{s_i}$ with $\sigma_{\mathtt{fd}_i} = \mathsf{user}$ for all $1 \leq i \leq n$, together with corresponding functions $f_{\mathtt{Nm}}$, $f_{\mathtt{fd}_1}, \ldots, f_{\mathtt{fd}_n}$. In terms of Sect. 3.2, `Nm` is a constructor and the $\mathtt{fd}_i$ are accessors.

`@UserOperation`. This is a method-level annotation to define "other operators," to be used on public methods in arbitrary public classes. All parameter types as well as the return type are either taken from the `java` algebra, *or* may be `@UserType`-annotated records. If the containing class is not a `@UserType`, the annotated method must be **`static`**. Hence there are two cases.

- ```
  @UserOperation
  public ty op(ty₁ fd₁, ..., tyₙ fdₙ)
  ```
  in a `@UserType`-annotated class defines an operator $\mathtt{op} \in O^{\mathtt{ty}}$ with $\sigma_{\mathtt{op}} = \mathsf{user}\, s_1 \cdots s_n$, together with a corresponding function $f_{\mathtt{op}}$.
- ```
  @UserOperation
  public static ty op(ty₁ fd₁, ..., tyₙ fdₙ)
  ```
  in *any* **`public`** class defines an operator $\mathtt{op} \in O^{\mathtt{ty}}$ with $\sigma_{\mathtt{op}} = s_1 \cdots s_n$, together with a corresponding function $f_{\mathtt{op}}$.

Table 1. Operators defined by the `Date` class of Fig. 1, with sorts and signatures. The $*$-marked operators are indeterminate

o	σ_o	s_o	
`Date`	`int int int`	`user`	
`day`	`user`	`int`	
`month`	`user`	`int`	
`year`	`user`	`int`	
`toString`	`user`	`string`	
`daysSince`	`user`	`int`	$*$
`next`	`user`	`user`	
`isLeapYear`	`int`	`bool`	
`today`	ϵ	`user`	$*$

Note that in the first of these cases, `op` has an extra argument of sort `user`: this corresponds to the **`this`**-value "on which," in object-oriented terminology, the method `op` is invoked. An example annotated class annotated is given in Fig. 1, defining the operators in Table 1. The intended meaning is self-explanatory.

To actually *use* annotation-defined operators, a rule system has to specify the qualified names of the annotated classes in its System Properties. From then on, they can be treated like built-in system operators.

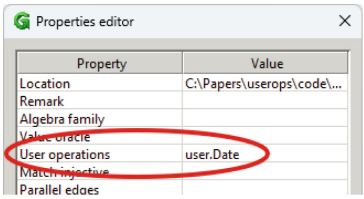


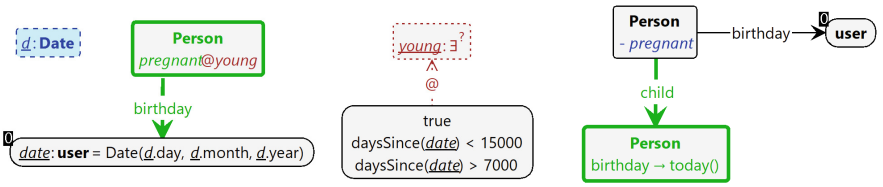
Figure 2 shows two example GROOVE rules using the operators of Fig. 1.

- create consumes a node d of (node) type **`Date`** and invokes the user-defined constructor `Date`, using d's node attributes as arguments. It then creates a node of type **`Person`** with that `Date` as their `birthday`. Moreover, the created **`Person`** is *`pregnant`* depending on a further condition, specified by the conditional quantifier `young`: this is satisfied if the number of days since the `birthday` is properly between 7000 and 15000.
- birth specifies that a *`pregnant`* person gives birth (and stops being *`pregnant`*). The newly born **`Person`** is a `child` of the first, and its `birthday` is based on the outcome of the `today` operator.

In both rules, the decoration 0 marks data nodes that serve as *rule parameters* (with number 0, meaning the first—here: only—parameter). Parameter values are part of the transition label when the rule gets applied. An example exploration is shown in Fig. 3: from the start graph on the left, using any of the rule application sequences of the transition system in the middle, eventually the final graph on the right is obtained. The indeterminacy of `today` causes the outcome of the exploration to depend on the day of execution.

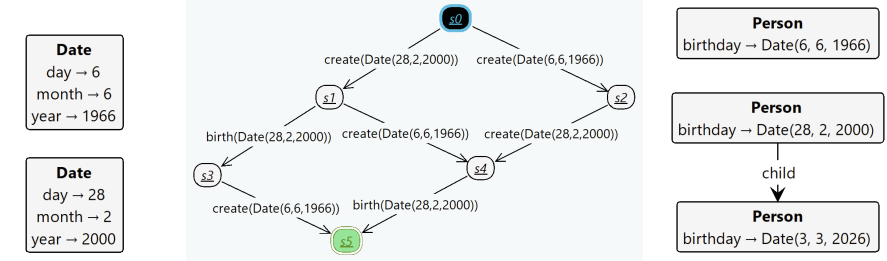
```
@UserType
public record Date(int day, int month, int year) {
   @UserOperation
   public String toString() { // Returns a String representation of this Date
      return "" + day() + "␣" + MONTHS[month() - 1] + "␣" + year();
   }

   @UserOperation(indeterminate = true)
   public int daysSince() { // Returns the number of days between this Date and now
      var cal = Calendar.getInstance(); cal.set(year(), month() - 1, day());
      long diff = Math.abs(new java.util.Date().getTime() - cal.getTime().getTime());
      return (int) TimeUnit.DAYS.convert(diff, TimeUnit.MILLISECONDS);
   }

   @UserOperation
   public Date next() { // Returns the next Date with respect to this one
      int day = day() + 1, month = month(), year = year();
      if (day > daysInMonth()) {
         day = 1; month += 1;
         if (month > 12) {
            month = 1; year += 1;
         }
      }
      return new Date(day, month, year);
   }

   private int daysInMonth() { // Returns the number of days in this Date's month
      return switch (month()) {
      case 1, 3, 5, 7, 8, 10, 12 -> 31;
      case 2 -> 28 + (isLeapYear(year()) ? 1 : 0);
      default -> 30;
      };
   }

   static private String[] MONTHS =
      {"Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov","Dec"};

   @UserOperation
   static public boolean isLeapYear(int year) { // Tests whether year is a leap year
      return year % 4 == 0 && (year % 100 != 0 || year % 400 == 0);
   }

   @UserOperation(indeterminate = true)
   static public Date today() { // Returns a Date object representing today
      var today = Calendar.getInstance();
      return new Date(today.get(Calendar.DAY_OF_MONTH),
           today.get(Calendar.MONTH) + 1, today.get(Calendar.YEAR));
   }
}
```

Fig. 1. Annotated `Date` class.



(a) create: a `Date`-typed node is transformed to a `Person` (b) birth: a *pregnant* `Person` gives birth

Fig. 2. GROOVE rules using the operators defined in Fig. 1.



(a) Start graph (b) Transition system (c) Final graph

Fig. 3. Example exploration based on the rules in Fig. 2.

5 Conclusion

We have presented an extension of GROOVE that enhances the tool's practical usefulness in incorporating data, based on user-defined record types and arbitrary operations. The extension addresses all the concerns identified in Sect. 2 (representation, immutability, equality, operations and randomness).

This work was inspired by a feature request.[1]

5.1 Limitations

We list some practical limitations in the current GROOVE implementation.

- *User types and operations are only supported in the* `java` *algebra.* In other words, the choice of algebras (`term`, `big`, `java`, `point`) recalled in the introduction is currently not available for grammars with user-defined types. We expect that the extension to `term` and `point` will be straightforward; however, to also be compatible with `big`, the user types and operations would natively have to rely on `BigInteger` and `BigDecimal` rather than **`int`** and **`double`**.
- *User types are restricted to* **`records`**. This is a choice motivated by the fact that Java **`record`** instances are inherently immutable and **`record`**s have well-defined constructors. We expect that this will not be a limitation in practice.

[1] Thanks to Reiko Heckel and Adam Machowczyk for their suggestions and feedback.

- *User type fields are restricted to primitive sorts.* Though there is no inherent problem in allowing user types to have user-typed fields, cyclic dependencies would necessitate the introduction of `null` values.
- *User type checking is weak.* The GROOVE type system cannot statically distinguish different (Java) user types: they are all values of the single `user` sort. Run-time type errors result in the special $\perp_{\texttt{user}}$ error value (see Sect. 3.1).
- *Operator names must be distinct.* The current implementation does not allow overloading. In particular, this means that also field names must be distinct across `@UserType` classes.
- *Indeterminate operations cannot appear in quantified rules.* For instance, it is not possible to simultaneously assign random values to a set of nodes by universally quantifying the nodes; or for the example in this paper, neither of the rules in Fig. 2 can be universally quantified so as to apply to all `Person` nodes simultaneously. Lifting this restriction will require a fairly major restructuring of GROOVE's rule matching engine.

5.2 Related Work

The seminal work of Ehrig et al. [3] embeds the theory of attributed graphs (in which actually not just nodes but also edges can have attributes) and their transformation into algebra. GROOVE's implementation is based on a variant of this embedding (see [4]); for practical purposes, the main difference is that only nodes can be attributed and that attribute types are restricted to the four primitive sorts recalled in Sect. 3.

Algebraic theory obviously allows much richer data types than just GROOVE's primitive attributes. However, once users are allowed to program their own types and operations, the equational reasoning that is foundational to algebra is very hard to uphold. This is also seen in the tool AGG[2] (see [8,11]): they, too, essentially import Java types into graphs, at the price of disabling some of AGG's analysis capabilities, e.g., critical pair analysis.

Again based on the theory of algebraic node and edge attributes, Lambers and Orejas et al. [9] have developed a very powerful theory of symbolic reasoning, which in that paper is reported as having been implemented in a tool called AutoGraph. (Unfortunately, the tool itself seems to be unavailable.)

A different way of integrating attributes and graphs has been both theoretically worked out and implemented by Plump et al. in the context of the Graph Programming tool GP (see [5,7]). Rather than aiming to be all-encompassing by allowing any algebraic type, they restrict data attributes to a compact, powerful but closed set of data types consisting of integers, strings and lists of those two, plus booleans.

[2] A more recent, open source snapshot of AGG can be found here.

On the other side of the spectrum, there are graph transformation tools that were conceived to be usable in practice, rather than to embrace theoretical concepts, and hence do not strive for a complete formalisation. Prime among these is Henshin (see, e.g., [1,10]), which actually shares many of GROOVE's capabilities regarding state space exploration and model checking. Citing [1], for data attributes they offer "flexible attribute computations based on Java or JavaScript."

Declaration of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Arendt, T., Biermann, E., Jurack, S., Krause, C., Taentzer, G.: Henshin: advanced concepts and tools for in-place EMF model transformations. In: Petriu, D.C., Rouquette, N., Haugen, Ø. (eds.) Model Driven Engineering Languages and Systems (MODELS), Part I. Lecture Notes in Computer Science, vol. 6394, pp. 121–135. Springer, Cham (2010). https://doi.org/10.1007/978-3-642-16145-2_9
2. Ehrig, H., Ehrig, K., Prange, U., Taentzer, G.: Fundamentals of Algebraic Graph Transformation. Monographs in Theoretical Computer Science. An EATCS Series, Springer, Heidelberg (2006). https://doi.org/10.1007/3-540-31188-2
3. Ehrig, H., Prange, U., Taentzer, G.: Fundamental theory for typed attributed graph transformation. In: Ehrig, H., Engels, G., Parisi-Presicce, F., Rozenberg, G. (eds.) Second International Conference on Graph Transformations (ICGT). LNCS, vol. 3256, pp. 161–177. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30203-2_13
4. Kastenberg, H., Rensink, A.: Graph attribution through sub-graphs. In: Heckel, R., Taentzer, G. (eds.) Graph Transformation, Specifications, and Nets — In Memory of Hartmut Ehrig. Lecture Notes in Computer Science, vol. 10800, pp. 245–265. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-75396-6_14
5. Manning, G., Plump, D.: The GP programming system. Electron. Commun. Eur. Assoc. Softw. Sci. Technol. **10** (2008). https://doi.org/10.14279/TUJ.ECEASST.10.150
6. Pierce, B.C.: Types and Programming Languages. MIT press (2002)
7. Plump, D.: The design of GP 2. In: Escobar, S. (ed.) 10th International Workshop on Reduction Strategies in Rewriting and Programming (WRS). EPTCS, vol. 82, pp. 1–16 (2011). https://doi.org/10.4204/EPTCS.82.1
8. Runge, O., Ermel, C., Taentzer, G.: AGG 2.0 - new features for specifying and analyzing algebraic graph transformations. In: Schürr, A., Varró, D., Varró, G. (eds.) Applications of Graph Transformations with Industrial Relevance - 4th International Symposium (AGTIVE). LNCS, vol. 7233, pp. 81–88. Springer, Cham (2011). https://doi.org/10.1007/978-3-642-34176-2_8
9. Schneider, S., Lambers, L., Orejas, F.: Automated reasoning for attributed graph properties. Int. J. Softw. Tools Technol. Transf. **20**(6), 705–737 (2018). https://doi.org/10.1007/S10009-018-0496-3

10. Strüber, D., et al.: Henshin: a usability-focused framework for EMF model transformation development. In: de Lara, J., Plump, D. (eds.) ICGT 2017. LNCS, vol. 10373, pp. 196–208. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-61470-0_12
11. Taentzer, G.: AGG: a graph transformation environment for modeling and validation of software. In: Pfaltz, J.L., Nagl, M., Böhlen, B. (eds.) AGTIVE 2003. LNCS, vol. 3062, pp. 446–453. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-25959-6_35

Author Queries

Chapter 12

Query Refs.	Details Required	Author's response
AQ1	As per Springer style, both city and country names must be present in the affiliations. Accordingly, we have inserted the city and country names in the affiliation. Please check and confirm if the inserted city and country names are correct. If not, please provide us with the correct city and country names.	